Project 1: Semantic Segmentation

이름	학번	학과	제출일
이승하	2021312779	소프트웨어학과	2026.05.05

1. 모델 아키텍처

구성	설명
[Backbone]	TorchVision EfficientNet-B1 의 ImageNet-1K classification weight 를 사전학습 가중치로 사용하였다.
[Neck]	DeepLabV3+를 따라, backbone 의 high-level feature(features[3:], OS=16, 1280ch)를 ASPP 에 통과시켜 다중 스케일 문맥 정보를 추출한다.
[SE Block]	ASPP 에서 생성된 다중 스케일 채널 중 입력에 따라 중요도가 높은 채널을 강조하기 위해 Squeeze-and-Excitation 블록을 적용하였다.
[**Decoder**]	DeepLabV3+를 따라, backbone 의 low-level feature(features[:3], stride=4, 24ch)를 1×1 Conv 로 48 채널까지 축소한 뒤, ×4 upsampling 한 ASPP 출력과 concat 하여 경계 정보를 보강한다.
[Head]	Decoder 가 정제한 feature 를 1×1 Conv 로 21 채널 logits 으로 투영하고, Bilinear Upsampling 으로 입력 해상도까지 복원해 최종 segmentation map 을 출력한다.

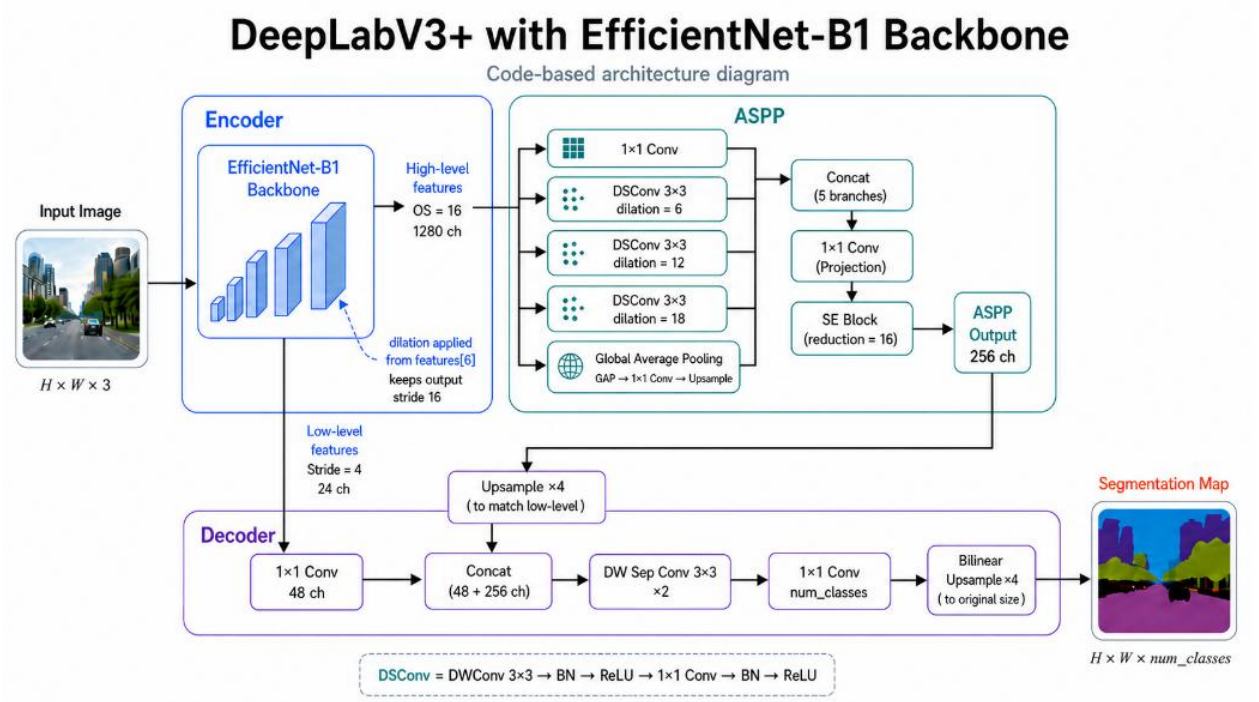


그림 1. 모델 아키텍처 다이어그램 (Input → Backbone → ASPP → Decoder → Mask)

2. 학습 레시피

2.1 데이터셋

[Training 데이터]	Pascal VOC 2007 & 2012 + MS-COCO VOC mapping.
[Validation 데이터]	Pascal VOC 2007 val & 2012 val.

2.2 최적화 설정

[항목]	[값]	[항목]	[값]
Optimizer	AdamW	Batch Size	48
Learning Rate	1.4e-3	Val Batch Size	16
Backbone LR Scale	0.05	Total Iters	110,000
Weight Decay	1.0e-4	Loss	CE + Dice + Lovasz (0.5)
Schedule	Poly + Warmup (300)	EMA Decay	0.9998
Seed	42	AMP	Enabled

최종 제출 모델인 V67 은 Loss Function 을 CE + Dice 로 설정하고 그 외의 모든 설정은 동일한 모델인 V7 의 70k iter 중간 체크포인트로 초기화한 뒤 Lovasz Sigmoid 를 loss 에 추가하여 fine-tuning 을 진행하였다. 제출물에는 V67 모델의 최종 체크포인트만 포함되어 있으며, 이 한 파일로 검증, 추론 결과를 그대로 재현할 수 있다.

2.3 데이터 증강

[Augmentation]	[확률]	[설명]	[적용범위]
Horizontal Flip	0.5	좌우반전	Image, mask
Random Rotation	1.0	±7 범위에서 회전	Image, mask
Random Scale	1.0	0.5×~1.75×	image=bilinear, mask=nearest
Pad to crop_size	조건부	crop_size(320) 미만일 때	image=0, mask=255
Random Crop	1.0	crop_size×crop_size (320)	
Gaussian Blur	0.3	kernel 5×5, $\sigma \in [0.1, 1.5]$	image
Color Jitter	1.0	명도 ±0.3, 대비 ±0.3, 채도 ±0.3, 색상 ±0.05	image
Copy-Paste	0.1	다른 샘플의 foreground (class≠0, ≠255)를 합성	image
Random Erasing	0.5	scale (0.02, 0.15), ratio (0.3, 3.3), 값=0	image

3. 검증 결과 및 실패 사례 분석

최종 검증 결과는 다음 명령어로 재현 가능하다.

```
python src/eval.py --config src/training_config.yaml --split val --use_ema
```

3.1 핵심 지표

Pixel Accuracy	95.43%	리더보드 mIoU	0.750
mIoU (EMA)	0.7940	리더보드 GFLOPs	24.957

3.2 클래스별 IoU

Class	IoU	Class	IoU	Class	IoU
[0] background	0.9488	[1] aeroplane	0.9298	[2] bicycle	0.4466
[3] bird	0.9270	[4] boat	0.7423	[5] bottle	0.6760
[6] bus	0.9552	[7] car	0.7921	[8] cat	0.9439
[9] chair	0.3955	[10] cow	0.9369	[11] dining table	0.5333
[12] dog	0.9042	[13] horse	0.9309	[14] motorbike	0.8543
[15] person	0.8952	[16] potted plant	0.6128	[17] sheep	0.9235
[18] sofa	0.5996	[19] train	0.9231	[20] tv monitor	0.8029

3.2 실패 사례 분석 (정량적)

IoU 가 낮게 나오는 원인을 세 가지 정도 가정해 보았다.

1. 클래스가 적게 등장해서 학습을 위한 데이터가 부족했다.
2. 해당 클래스가 이미지에서 작게 등장해서 학습이 어려웠다.
3. 해당 클래스가 등장할 때 다른 클래스들도 함께 등장해 장면이 복잡했다.

세 가지 가정이 맞는지 확인하기 위해 Claude 를 활용하여 데이터셋 분석을 수행해보았다.

	class	image_count	image_freq_%	mean_area_%	mean_co_classes	mean_bg_%
0	background	96901	99.947397	76.475901	1.690963	76.475901
1	aeroplane	3086	3.183018	12.455743	1.557356	85.940117
2	bicycle	3328	3.432626	5.646439	2.893029	78.766125
3	bird	3355	3.460475	5.834609	1.645306	88.178689
4	boat	3114	3.211899	10.001299	2.123314	84.766319
5	bottle	8605	8.875526	2.605774	2.534922	72.101891
6	bus	4044	4.171136	21.479837	2.635757	71.328688
7	car	12393	12.782614	5.940357	2.408376	80.946880
8	cat	4260	4.393927	18.757701	1.835211	66.543332
9	chair	12948	13.355062	5.725012	2.886778	69.463358
10	cow	2043	2.107228	15.095458	1.727362	80.574437
11	diningtable	11933	12.308152	31.541438	2.526355	51.556902
12	dog	4523	4.665195	12.608239	2.204952	72.374465
13	horse	3024	3.119069	13.573579	2.096230	79.711910
14	motorbike	3594	3.706989	15.514427	2.598776	72.848874
15	person	64649	66.681451	15.897134	1.825055	76.224330
16	pottedplant	4551	4.694075	6.518725	2.912547	75.059725
17	sheep	1600	1.650301	14.193040	1.581250	81.719494
18	sofa	4533	4.675510	15.276964	3.087801	62.991452
19	train	3679	3.794661	23.824557	1.707529	73.276231
20	tvmonitor	4664	4.810628	8.930466	2.750214	74.501847

등장 빈도와 평균 객체 크기는 낮은 IoU 를 단독으로 설명하기 어려웠다. 예를 들어 chair 와 dining table 은 비교적 자주 등장하지만 IoU 가 낮았고, dining table 은 평균 객체 크기가 가장 큰 편임에도 성능이 낮았다.

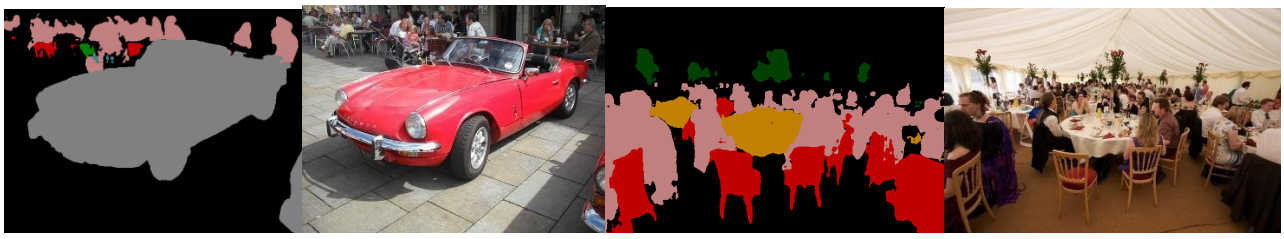
세 가설 중 가장 설득력 있는 요인은 장면 혼잡도였다. 낮은 IoU 를 보인 클래스들은 평균 2.5 개의 다른 foreground class 와 등장했다. 이는 여러 클래스가 함께 등장하는 복잡한 장면에서 클래스 혼동이 증가했을 가능성을 보여준다.

3.3 실패 사례 분석 (정성적)

Predict.py 를 통해 제공된 이미지를 변환해본 뒤 원본 이미지와 비교를 해보았다.

Chair, Dining Table, Bicycle 같은 IoU 가 낮은 이미지들을 보았을 때 얇은 부분을 가진 물체들의 경계를 찾는 것이 어려웠다는 것을 알 수 있었다.

potted plant 클래스에서는 물체의 재질에 따른 품질 변화를 관측할 수 있었다. 투명한 유리 화분은 배경과 구분이 힘들었지만 불투명한 재질의 화분은 상대적으로 경계가 안정적으로 나왔다. 이를 통해 물체의 재질 또한 segmentation 품질에 영향을 미친다는 것을 알 수 있었다.



두 이미지를 보았을 때 상대적으로 IoU 가 높은 person 클래스여도 이미지 내에 작은 객체들이 다수 존재하는 상황이라면 그 경계가 무너지고 정확도가 떨어지는 모습을 보았다.

4. 실험 이력 및 Ablation

실행	역할	제출	iter	Best mIoU (EMA)	비고
V5_2	EfficientNet-B0 baseline	X	94k	0.6813 (80k)	CE+Dice, VOC+COCO, EMA, AMP
V6	Loss ablation	X	80k	0.6865 (70k)	V5_2 기반 Lovasz Loss 추가, Batch size 32 -> 48 로 변경.
V7	EfficientNet-B1	X	100k	0.7031 (70k)	V5_2 기반, backbone B1 으로 변경. Batch size 32 -> 48 로 변경.
V67	[object Object]	O	110k	0.7130 (110k)	V7 의 70k 에서 시작하며, V6 의 loss ablation 반영. backbone_lr_scale 0.15 -> 0.05 로 변경.

4.1 핵심 관찰

1. EfficientNet B1 backbone 선택

초기 모델은 MobileNetV2 backbone 과 DeepLabV3+ 구조를 사용하였다. 그러나 강한 augmentation 을 적용해도 성능이 일정 수준 이상으로 향상되지 않는 한계가 있었다. 이에 따라 더 높은 표현력을 가진 backbone 이 필요하다고 판단하여 EfficientNet-B0 기반의 V5_2 모델을 구성하였다. 이후 FLOPs 제한에 아직 여유가 있었기 때문에, 성능 향상을 위해 EfficientNet-B1 backbone 을 적용한 V7 모델을 추가로 실험하였다. 그 결과 V7 은 V5_2 보다 validation mIoU 와 제출 사이트 점수 모두에서 더 좋은 성능을 보였고, 최종 모델의 기반 구조로 선택하였다.

2. EMA 가중치.

EMA 는 학습 중 모델 파라미터의 지수 이동 평균을 유지하는 방법으로, 파라미터 변동을 완화하여 더 안정적인 추론 성능을 기대할 수 있다. 본 프로젝트에서는 수업에서 배운 parameter smoothing 개념을 모델 학습에 적용하기 위해 EMA 가중치를 사용하였다. 최종 제출에서는 validation 성능이 가장 좋았던 EMA checkpoint 를 사용하였다.

3. Lovasz Loss.

CE loss 는 각 픽셀이 어떤 클래스인지 맞추는 데 초점을 두고, Dice loss 는 예측한 객체 영역과 실제 객체 영역이 더 많이 겹치도록 돕는다. 하지만 최종 평가 지표인 mIoU 를 직접 최적화하지는 않는다. 추가적인 성능 개선 방법을 검토하는 과정에서 IoU 를 직접 반영하는 대체 손실 함수인 Lovasz-SoftMax loss 를 적용하였다. 먼저 V5_2 기반 모델에 Lovasz loss 를 추가한 V6 를 실험한 결과 mIoU 가 개선되는 것을 확인하였다. 이후 EfficientNet-B1 기반으로 학습 중이던 V7 모델에도 Lovasz loss 를 추가하여 V67 모델을 구성하였고, 이를 최종 제출 모델로 선택하였다.

5. Wandb 그래프 및 로그

5.1 모니터링 및 학습 곡선 (V5_2, V6, V7, V67)

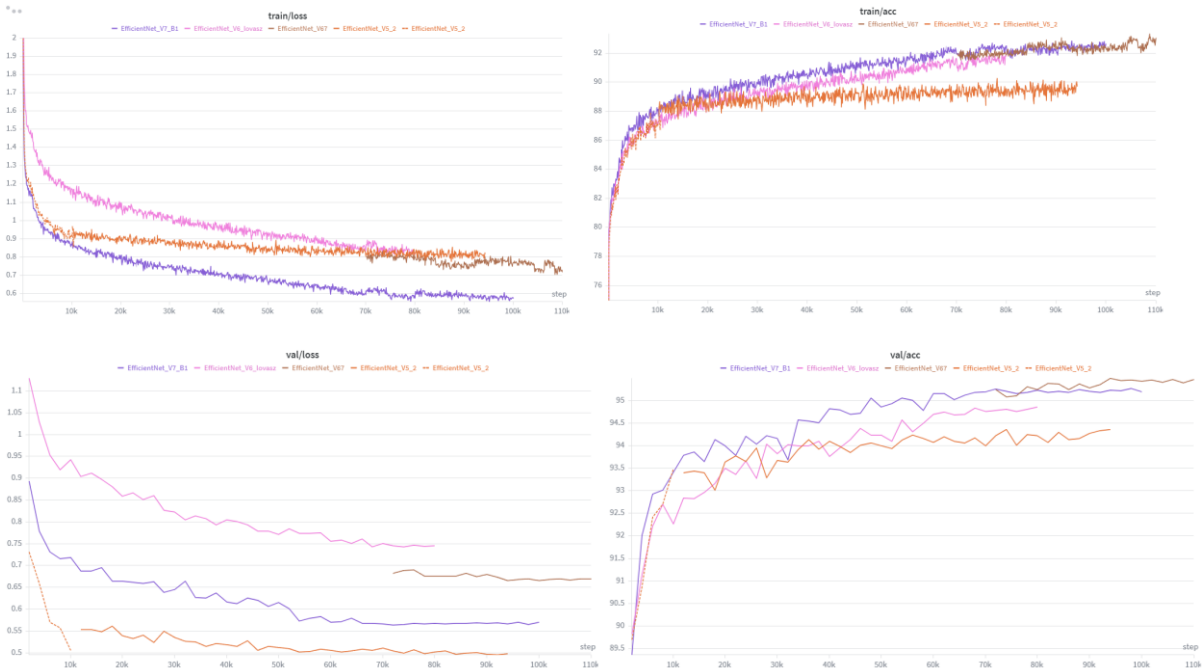


그림 5. W&B Charts — train/loss, train/acc, val/loss, val/acc

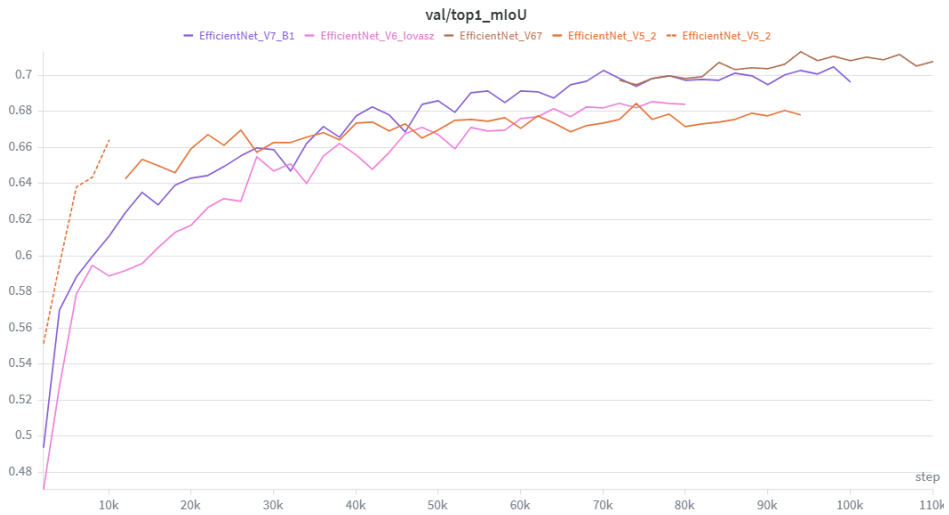


그림 6. W&B Charts — val/top1_mIoU

6.2 Config / Overview

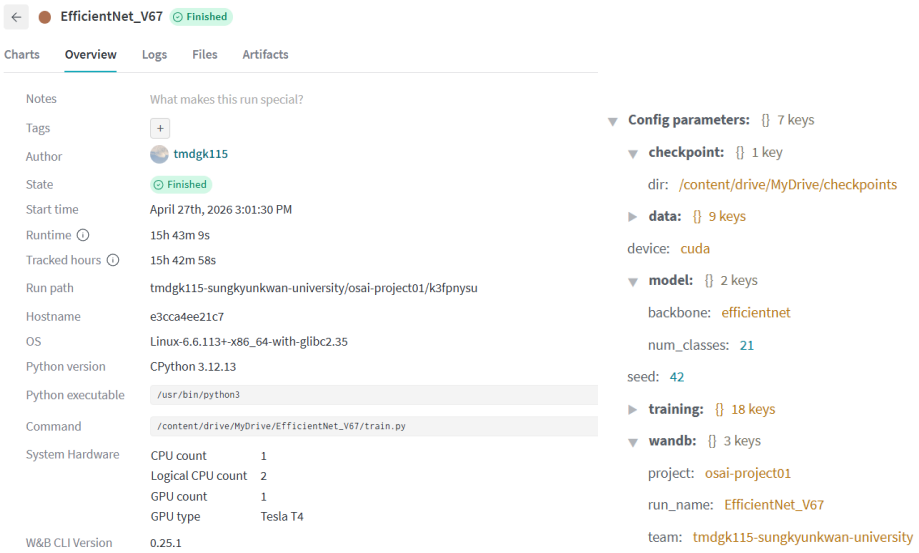


그림 7, 8. W&B Overview · Config 페이지 — 모델, 데이터, 시드, 하드웨어 정보 (training 정보는 학습 레시피에 존재)

6.3 AI 도구 사용

AI 도구는 프로젝트 수행 과정에서 보조적으로 활용하였다. 모델 구조, augmentation, loss, optimizer 선택 및 최종 모델 결정은 직접 수행하였으며, 코드 작성과 세부 구현 과정에서는 ChatGPT/Codex 의 도움을 받았다. 또한 AI 모델 관련 프로젝트 경험이 부족했기 때문에 hyperparameter 조정 과정에서 AI 의 조언을 참고하였다. 보고서 작성에서는 Claude Design 을 사용해 전체 양식과 가독성 있는 구조를 구성하였고, ChatGPT/Codex 는 문장 검수와 표현 정리에 활용하였다.

참고문헌

[1] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, and H. Adam, "Encoder-Decoder with Atrous Separable Convolution for Semantic Image Segmentation," ECCV, 2018. arXiv:1802.02611.

[2] M. Berman, A. R. Triki, and M. B. Blaschko, "The Lovász-Softmax Loss: A Tractable Surrogate for the Optimization of the Intersection-over-Union Measure in Neural Networks," CVPR, 2018. arXiv:1705.08790.